

Experiment – 6

Name: Renulasya

UID: 24BAI70210

Section/Group: 24AIT-NTTP-2[A]

Branch: CSE – AIML (CS221) /5th

Subject Code: 24CSP-337

Subject: Full Stack - II

Date of Performance:

Submitted to: Ms. Loveleen Kaur (E20454)

EXPERIMENT – 6.1

1. **AIM:** To implement paginated and sortable REST APIs for efficient and scalable data retrieval.

2. **OBJECTIVE:**

1. To understand pagination and sorting concepts
2. To implement pageable APIs using Spring Data
3. To optimize data retrieval for large datasets
4. To improve API performance and usability

3. **SOFTWARE REQUIREMENTS:**

1. Java JDK 17
2. Spring Boot
3. IDE (IntelliJ / Eclipse / VSCode)
4. Postman / API testing tool

4. **THEORY**

In real-world applications, datasets can grow significantly large, making it inefficient to retrieve all records in a single API call. Pagination addresses this issue by dividing data into smaller chunks (pages), allowing clients to request only a subset of data at a time.

Sorting enhances usability by allowing data to be ordered based on specific fields such as creation date, popularity, or relevance.

Spring Boot provides built-in support for pagination and sorting through the Pageable interface, which simplifies API design and ensures efficient query execution.

Benefits of pagination and sorting include:

- Reduced server load
- Faster response times
- Improved user experience
- Better scalability

5. IMPLEMENTATION/PROCEDURE

1. Create a Spring Boot project.
2. Add Spring Web and Spring Data JPA dependencies.
3. Configure the database.
4. Create the required Entity class.
5. Create a Repository using JpaRepository.
6. Create a Service layer.
7. Implement pagination using Pageable.
8. Implement sorting using Sort.
9. Create a REST Controller.
10. Add page number and page size parameters to the API.
11. Add sorting parameters such as field name and direction.
12. Run the Spring Boot application.
13. Open the application in a web browser.
14. Verify that records are displayed page by page.
15. Verify that records can be sorted in ascending and descending order.

6. CODE

Controller

```
package package com.example.pagination_api.controller;  
import com.example.pagination_api.model.Post;  
import com.example.pagination_api.service.PostService;
```

```
import org.springframework.data.domain.Page;  
import org.springframework.data.domain.PageRequest;  
import org.springframework.data.domain.Pageable;  
import org.springframework.data.domain.Sort;
```

```
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;
```

```
@RestController
```

```
@RequestMapping("/api/posts")
```

```
@CrossOrigin(origins = "*")
```

```
public class PostController {
```

```
    private final PostService postService;
```

```
    public PostController(PostService postService) {  
        this.postService = postService;  
    }
```

```
    // GET posts with pagination and sorting
```

```
    @GetMapping
```

```
    public ResponseEntity<Page<Post>> getAllPosts(  
  
        @RequestParam(defaultValue = "0") int page,  
  
        @RequestParam(defaultValue = "5") int size,
```

```
@RequestParam(defaultValue = "id") String sortBy,

@RequestParam(defaultValue = "asc") String direction) {

Sort sort;

if (direction.equalsIgnoreCase("desc")) {
    sort = Sort.by(sortBy).descending();
} else {
    sort = Sort.by(sortBy).ascending();
}

Pageable pageable = PageRequest.of(page, size, sort);

return ResponseEntity.ok(
    postService.getAllPosts(pageable)
);
}

// POST - create a new post
@PostMapping
public ResponseEntity<Post> createPost(@RequestBody Post post) {

    return ResponseEntity.ok(
        postService.createPost(post)
    );
}
}

package com.example.pagination_api.service;

import com.example.pagination_api.model.Post;
import com.example.pagination_api.repository.PostRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

@Service
public class PostService {

    private final PostRepository postRepository;

    public PostService(PostRepository postRepository) {
        this.postRepository = postRepository;
    }

    // Get posts with pagination and sorting
    public Page<Post> getAllPosts(Pageable pageable) {
        return postRepository.findAll(pageable);
    }

    // Create a new post
    public Post createPost(Post post) {
        return postRepository.save(post);
    }
}
```

7. OUTPUT

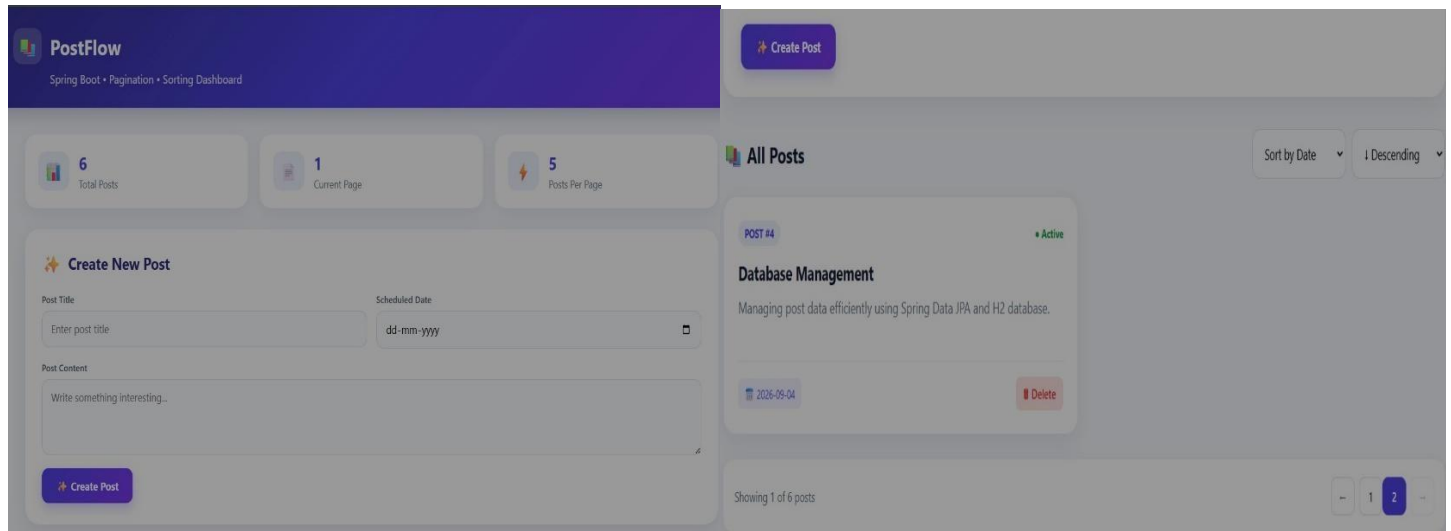


Fig 1: Create and CRUD Operations Interface

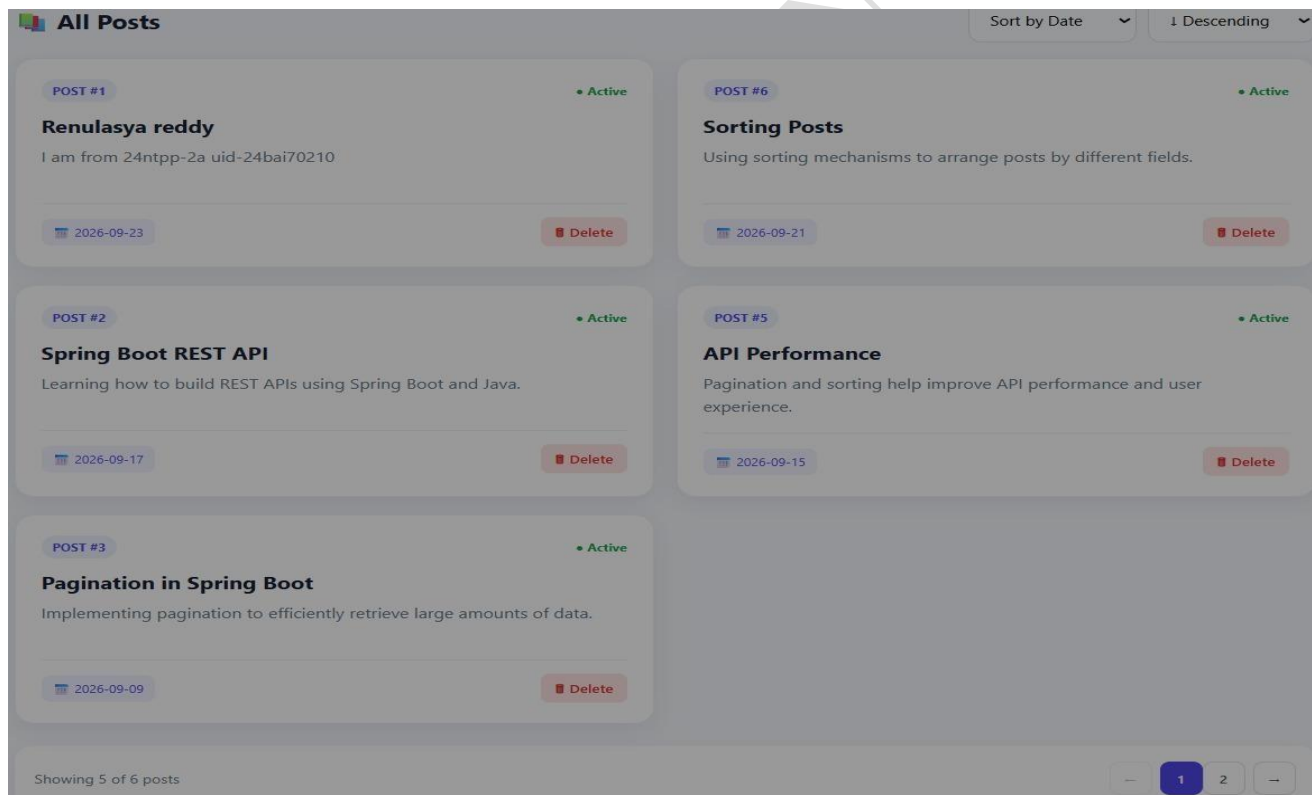


Fig 2: Posts Listing, Sorting and Pagination Interface

8. LEARNING OUTCOME

- Understand pagination and sorting.
- Implement Pageable in Spring Data JPA.
- Implement sorting using Spring Data.
- Retrieve large datasets efficiently.

EXPERIMENT – 6.2

1. **AIM:** To optimize backend read operations using caching and efficient database query strategies.

2. **OBJECTIVE:**

1. To identify and resolve inefficient database queries.
2. To understand the N+1 query problem.
3. To implement caching mechanisms.
4. To improve API performance through optimization

3. **SOFTWARE REQUIREMENTS:**

1. Java JDK 17+
2. Spring Boot
3. Visual Studio Code / IntelliJ / Eclipse
4. Maven
5. SLF4J / Logback
6. Web Browser

4. **THEORY**

Backend performance is important when an application handles a large number of requests. Repeated database queries and inefficient queries can increase response time and database load.

The **N+1 query problem** occurs when one query retrieves a set of records and additional queries are executed for related records. This results in unnecessary database operations.

The problem can be reduced using **JOIN FETCH**, which retrieves related data using a single optimized query and reduces database communication.

Caching is another important optimization technique. Frequently accessed data is stored temporarily in memory so that repeated requests can be served faster without querying the database every time.

Spring Boot provides caching annotations such as:

- `@EnableCaching` – enables caching.
- `@Cacheable` – stores method results in the cache.
- `@CacheEvict` – removes data from the cache.
- `@CachePut` – updates the cache.

Native SQL queries can be used for complex database operations when more control over the SQL query is required.

Apache JMeter can be used to test API performance under different loads and compare response times before and after optimization.

5. IMPLEMENTATION/PROCEDURE

1. Create a Spring Boot project with Spring Data JPA.
2. Configure the database connection.
3. Create the required entity classes and database relationships.
4. Create the Repository and Service layers.
5. Identify frequently accessed read operations.
6. Identify inefficient database queries and understand the N+1 query problem.
7. Use JOIN FETCH to optimize related-entity queries.
8. Enable caching using `@EnableCaching`.
9. Apply `@Cacheable` to frequently accessed read methods.
10. Use `@CacheEvict` when data is created, updated or deleted.
11. Implement native SQL queries for complex operations where required.
12. Run the Spring Boot application.
13. Test the APIs through the web interface/browser.
14. Use Apache JMeter to send multiple requests and observe API performance.
15. Compare the performance before and after applying caching and query optimization.

6. CODE

Controller

```
package com.example.cache_api.controller;
import com.example.cache_api.model.Post;
import com.example.cache_api.service.PostService;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/posts")
public class PostController {
    private final PostService postService;

    public PostController(PostService postService) {
        this.postService = postService;
    }

    @GetMapping
    public List<Post> getAllPosts() {

        return postService.getAllPosts();
    }

    @GetMapping("/{id}")
    public Optional<Post> getPostById(@PathVariable
Long id) {
        return postService.getPostById(id);
    }

    @PostMapping
    public Post createPost(@RequestBody Post post) {
```

```
        return postService.createPost(post);
    }
    @PutMapping("/{id}")
    public Post updatePost(
        @PathVariable Long id,
        @RequestBody Post post) {

        return postService.updatePost(id, post);
    }
    @DeleteMapping("/{id}")
    public String deletePost(@PathVariable Long id) {

        postService.deletePost(id);

        return "Post deleted successfully";
    }
}

package com.example.cache_api.service;

import com.example.cache_api.model.Post;
import
com.example.cache_api.repository.PostRepository;

import
org.springframework.cache.annotation.CacheEvict;
import
org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class PostService {

    private final PostRepository postRepository;

    public PostService(PostRepository postRepository) {
        this.postRepository = postRepository;
    }

    @Cacheable("posts")
    public List<Post> getAllPosts() {

        return postRepository.findPostsWithJoinFetch();
    }

    @Cacheable(value = "post", key = "#id")
```

7. OUTPUT

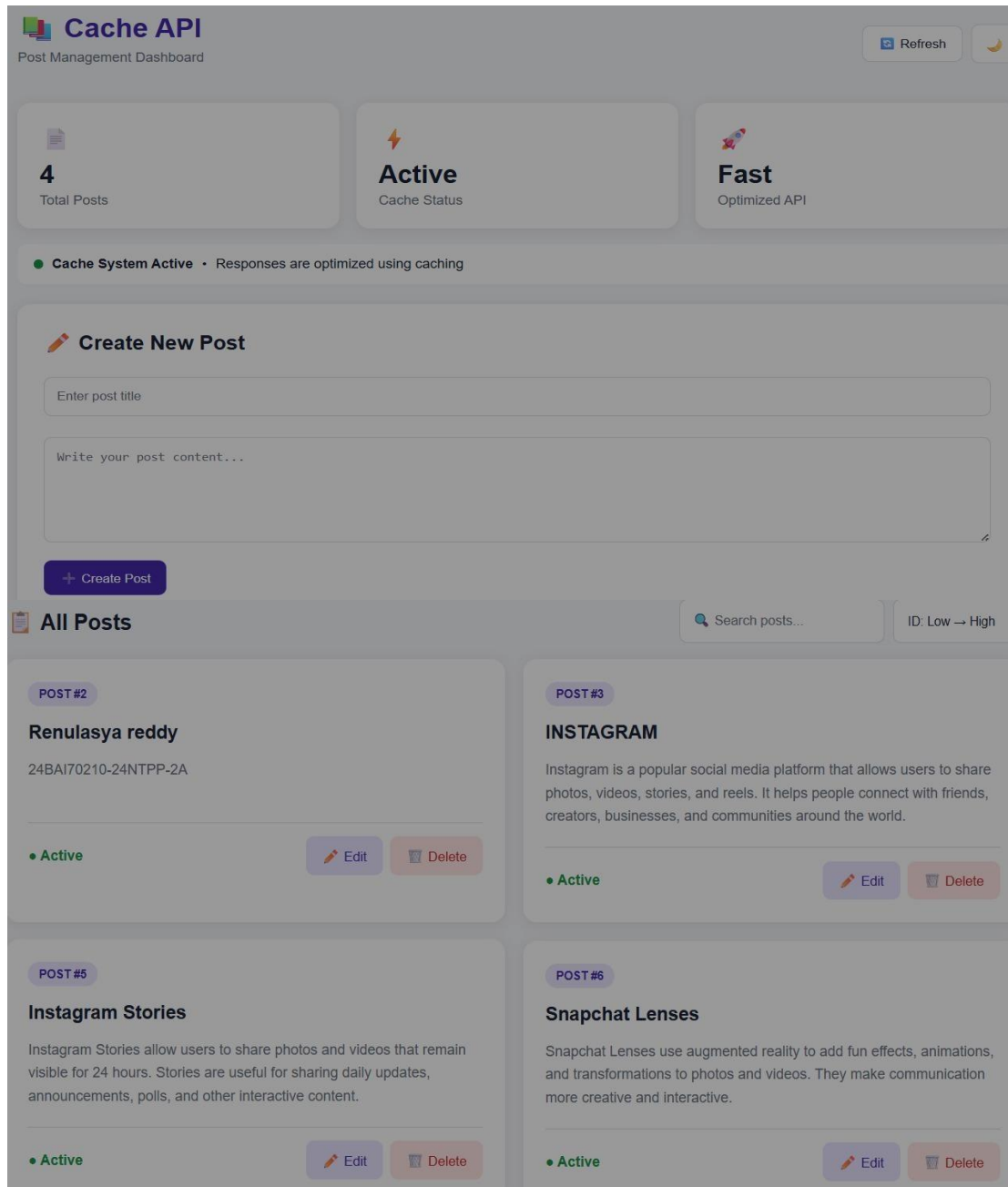


Fig 1.1: All Posts Dashboard with Search, Sort, Edit and Delete Features

8. LEARNING OUTCOME

- Understood backend performance optimization.
- Identify the N+1 query problem.
- Understand and apply JOIN FETCH.
- Implement caching in Spring Boot.
- Use caching annotations effectively.
- Write optimized database queries.